

Reach-Avoid Differential Game for UAV Interception

Implementation Documentation

Based on the paper by Bui, Monckton, and Chen
*“Reach-Avoid Differential Game with Reachability Analysis
for UAVs — A Decomposition Approach”*
(arXiv:2512.22793)

Framework:	Hamilton-Jacobi Reachability Analysis
Solver:	<code>hj_reachability</code> v0.7.0 (JAX)
Simulation:	Gazebo Harmonic + PX4 SITL
Hardware:	Crazyflie 2.1 + Crazyswarm2
Middleware:	ROS 2 Humble

March 16, 2026

Contents

1	Project Overview	3
1.1	Key Results	3
2	Mathematical Foundation	3
2.1	Dynamics Modeling	3
2.2	Game Decomposition	4
2.3	Hamilton-Jacobi Reachability	4
2.4	Invariant Capture Sets	4
2.5	Computation Pipeline	5
2.6	Capture Guarantee Analysis	5
3	Implementation Architecture	5
3.1	Repository Structure	5
3.2	Offline Computation Package: <code>reach_avoid_game</code>	6
3.2.1	Dynamics Classes	6
3.2.2	Solver Functions	6
3.2.3	Control Extraction	6
3.2.4	Winning Conditions	7
3.2.5	Value Function I/O	7
3.3	Online Control: ROS2 Workspace	8
3.3.1	Package Overview	8
3.3.2	Defender Controller Node	8
3.3.3	ROS2 Topic Architecture	9
3.4	Configuration	9
4	Running the Offline Computation	10
4.1	Prerequisites	10
4.2	Computing Value Functions	10
4.2.1	Step 1: Vertical Pipeline	10
4.2.2	Step 2: Horizontal Pipeline	10
4.3	Running Numerical Simulations	11
4.4	Generating Paper Figures	11
4.5	Running Tests	11
5	Running the Gazebo Simulation	11
5.1	Prerequisites	11
5.2	Docker Setup	12
5.3	Launch Commands	12
5.4	Gazebo World	13
5.5	Simulation Parameters	13
6	Running on Real Drones	13
6.1	Hardware Requirements	13
6.2	Software Setup	13
6.3	Crazyflie Configuration	14
6.4	Pre-Flight Checklist	14
6.5	Launch Sequence	14
6.6	Safety Monitor	15
6.7	Crazyswarm2 Adapter	15

6.8	Tuning for Real Hardware	16
7	Troubleshooting	16
8	References	16

Project Overview

This project implements a **3D reach-avoid differential game** between two unmanned aerial vehicles (UAVs). One drone, the **defender**, attempts to intercept a second drone, the **attacker**, before the attacker reaches a designated target region. The attacker simultaneously tries to navigate to its goal while avoiding both obstacles and the defender.

The core challenge is that computing optimal strategies for two drones in 3D space would require solving a Hamilton-Jacobi (HJ) partial differential equation on a **24-dimensional** joint state space (12 dimensions per quadrotor). This is computationally intractable. The paper introduces a **decomposition framework** that splits the problem into two solvable sub-games:

1. **Horizontal sub-game (6D):** Models the x-y plane dynamics of both drones, including obstacles and the target region.
2. **Vertical sub-game (3D):** Models the z-axis dynamics of both drones.

Each sub-game is solved independently using HJ reachability analysis, and the solutions are recombined using a novel **reach-track control algorithm** that provides provable capture guarantees.

The implementation spans three layers:

- **Offline computation:** Value functions are solved on a discretized grid using the `hj_reachability` library (JAX/GPU-accelerated). These represent optimal strategies for all possible states.
- **Online control:** A real-time ROS2 controller reads drone states, interpolates precomputed value functions, extracts optimal controls via gradient computation, and publishes velocity commands at 50 Hz.
- **Simulation & hardware:** The game runs in Gazebo with PX4 autopilot for physics-based validation, or on real Crazyflie 2.1 drones with motion capture for state estimation.

Key Results

- The decomposition reduces the problem from 24D (intractable) to a 6D + 3D pair (solvable in minutes to hours).
- Formal capture guarantees: if the initial state is in the defender's **winning region**, the defender is guaranteed to capture the attacker regardless of the attacker's strategy.
- The solution is **conservative**: the decomposition's winning region is a subset of the true optimal winning region, meaning the defender never falsely claims it can win.

Mathematical Foundation

Dynamics Modeling

Both drones are simplified from the full 12D quadrotor dynamics:

Definition 1 (Defender — Double Integrator). *The defender has inertia (cannot change velocity instantaneously):*

$$\dot{x}_D = v_D, \quad \dot{v}_D = K(u - v_D) \quad (1)$$

where u is the velocity command and $K = \text{diag}(k_x, k_y, k_z)$ are proportional gains. The state is 6D: position (x_D, y_D, z_D) and velocity $(v_{D,x}, v_{D,y}, v_{D,z})$.

Definition 2 (Attacker — Single Integrator). *The attacker has no inertia (controls velocity directly):*

$$\dot{x}_A = d \quad (2)$$

where d is the attacker's velocity command. The state is 3D: position (x_A, y_A, z_A) . This **overestimates** the attacker's capability, making the defender's strategy more conservative.

Speed constraints:

$$\|(u_x, u_y)\| \leq U_D^h = 6 \text{ m/s}, \quad \|(d_x, d_y)\| \leq U_A^h = 3 \text{ m/s} \quad (3)$$

$$|u_z| \leq U_D^z = 4 \text{ m/s}, \quad |d_z| \leq U_A^z = 2 \text{ m/s} \quad (4)$$

The defender is twice as fast as the attacker in all directions.

Game Decomposition

The capture condition is decomposed from a sphere into a **cylinder**:

- **Original:** $\|p_A - p_D\|_2 \leq d_c$ (spherical capture zone)
- **Decomposed:**

$$\textbf{Horizontal: } \sqrt{(x_A - x_D)^2 + (y_A - y_D)^2} \leq d_h = 3 \text{ m} \quad (5)$$

$$\textbf{Vertical: } |z_A - z_D| \leq d_z = 1 \text{ m} \quad (6)$$

Key assumption: The target region $\mathcal{T}$ and obstacles depend only on horizontal (x, y) coordinates, not altitude. This is practical for airspace corridor scenarios.

Hamilton-Jacobi Reachability

For each sub-game, we solve the HJ variational inequality backward in time:

$$\min \left\{ \frac{\partial V}{\partial t} + H(x, \nabla_x V), V(x, t) - l(x) \right\} = 0 \quad (7)$$

where the Hamiltonian is:

$$H(x, p) = \max_u \min_d p \cdot f(x, u, d) \quad (8)$$

The defender maximizes (tries to capture) and the attacker minimizes (tries to escape). The **zero level set** $\{x : V(x) \leq 0\}$ defines the winning region.

Invariant Capture Sets

Simply combining the two sub-game solutions does not guarantee simultaneous capture in both directions. The defender might overshoot vertically due to inertia while capturing horizontally. The solution uses **invariant capture sets**:

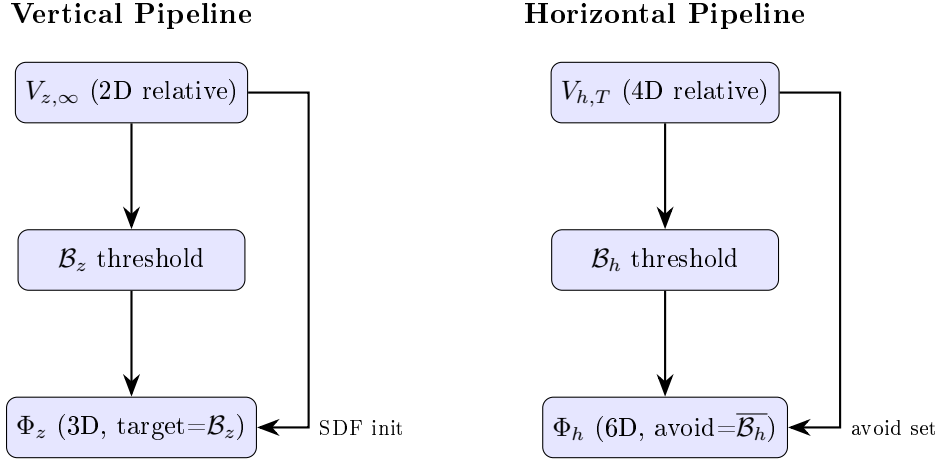
Definition 3 (Invariant Capture Set $\mathcal{B}_z$). $\mathcal{B}_z = \{x : V_{z,\infty}(x) \leq d_z\}$ where $V_{z,\infty}$ is the maximum distance value function. Once inside $\mathcal{B}_z$, the defender can keep $|z_D - z_A| \leq d_z$ **forever** using a tracking controller.

Definition 4 (Invariant Capture Set $\mathcal{B}_h$). $\mathcal{B}_h = \{x : V_{h,T}(x) \leq d_h\}$ where $V_{h,T}$ is computed with $T = 2.5 \text{ s}$ (does not converge as $T \rightarrow \infty$ due to the horizontal dynamics structure).

These feed back into the reach-avoid computation: Φ_z uses $\mathcal{B}_z$ as its target set (Eq. 38 of the paper), and Φ_h uses the complement of $\mathcal{B}_h$ as part of its avoid set (Eq. 39).

Computation Pipeline

The correct computation order (matching the paper) is:



Capture Guarantee Analysis

Theorem 1 (Defender Wins). *The defender is guaranteed to capture the attacker if all of the following hold:*

1. $x^z \in \mathcal{W}_{D,z}$ — vertical state is in defender's vertical winning region
2. $T_{goal} > T_{capture}$ — defender captures vertically before attacker reaches goal
3. $x^h \in \mathcal{W}_{D,h}$ — horizontal state is in defender's horizontal winning region

Where T_{goal} is the earliest time the attacker can reach its target (from $\Phi_{A,reach}$) and $T_{capture}$ is the earliest time the defender achieves vertical capture (from Φ_z time slices).

Implementation Architecture

Repository Structure

Top-Level Directory Layout

```

/workspace/
  config/
    game_params.yaml      # Game parameters and grid presets
  data/
    value_functions/      # Precomputed .npz value function files
    plots/paper_figures/  # Generated paper figures
    simulations/           # Simulation trajectory outputs
  reach_avoid_game/      # Core Python package (offline computation)
    src/reach_avoid_game/
      config.py           # GameConfig dataclass
      dynamics/           # HJ dynamics classes
      solvers/            # HJ solvers and utilities
      visualization/      # Plotting functions
    scripts/              # CLI entry points
    tests/                # Pytest test suite (60 tests)
  reach_avoid_ws/         # ROS 2 workspace (online control)
    src/
      reach_avoid_bringup/ # Launch files and parameters

```

```

reach_avoid_sim/      # Gazebo adapters, PX4 bridge
reach_avoid_controller/ # Defender reach-track controller
attacker_controller/  # Attacker control modes
reach_avoid_viz/      # RViz MarkerArray visualization
reach_avoid_hw/       # Crazyflie hardware interface

```

Offline Computation Package: reach_avoid_game

3.2.1 Dynamics Classes

Each dynamics class inherits from `hj.ControlAndDisturbanceAffineDynamics` and implements three methods:

Table 1: Dynamics classes and their state spaces.

Class	Dim	State	File
VerticalGameDynamics	3D	$(z_D, v_{D,z}, z_A)$	vertical_game.py
VerticalRelativeDynamics	2D	$(z_{\text{rel}}, v_{D,z})$	vertical_relative.py
HorizontalGameDynamics	6D	$(x_D, y_D, v_{D,x}, v_{D,y}, x_A, y_A)$	horizontal_game.py
HorizontalRelativeDynamics	4D	$(x_{\text{rel}}, y_{\text{rel}}, v_{D,x}, v_{D,y})$	horizontal_relative.py
HorizontalGameTrackingDynamics	6D	same as above	horizontal_game.py
AttackerReachingDynamics	2D	(x_A, y_A)	attacker_reaching.py

Each class provides:

- `open_loop_dynamics(state, time)` — drift dynamics $f(x)$ when $u = 0, d = 0$
- `control_jacobian(state, time)` — matrix G_u mapping control to state derivatives
- `disturbance_jacobian(state, time)` — matrix G_d mapping disturbance to state derivatives

The `HorizontalGameTrackingDynamics` class is identical to `HorizontalGameDynamics` except with **swapped optimization**: `control_mode="min"` (defender minimizes distance) and `disturbance_mode="max"` (attacker maximizes distance). This is used for computing the maximum distance value function $V_{h,T}$ in 6D coordinates with obstacle penalties.

3.2.2 Solver Functions

All solvers are in the `solvers/` subdirectory:

3.2.3 Control Extraction

Optimal controls are extracted from value function gradients using central finite differences:

$$\nabla V(x) \approx \left[\frac{V(x + h e_i) - V(x - h e_i)}{2h} \right]_{i=1}^n \quad (9)$$

The defender's optimal control (maximizing the value function):

$$u_z^* = U_{D,z} \cdot \text{sign} \left(\frac{\partial V}{\partial v_{D,z}} \cdot k_z \right) \quad (10)$$

The attacker's optimal disturbance (minimizing the value function):

$$d_z^* = -U_{A,z} \cdot \text{sign} \left(\frac{\partial V}{\partial z_A} \right) \quad (11)$$

Table 2: Value functions computed by each solver function.

Function	Output	Grid	Description
<code>solve_vertical_max_distance</code>	$V_{z,\infty}$	2D	Max distance tracking (converged)
<code>compute_invariant_set_Bz</code>	$\mathcal{B}_z$	2D	Threshold $V_{z,\infty} \leq d_z$
<code>solve_vertical_reach_avoid</code>	Φ_z	3D	Vertical reach-avoid with $\mathcal{B}_z$ target
<code>solve_horizontal_max_distance</code>	$V_{h,T}$	4D	Max distance tracking ($T = 2.5$ s)
<code>compute_invariant_set_Bh</code>	$\mathcal{B}_h$	4D	Threshold $V_{h,T} \leq d_h$
<code>solve_attacker_reaching</code>	$\Phi_{A,\text{reach}}$	2D	Attacker time-to-goal
<code>solve_horizontal_reach_avoid</code>	Φ_h	6D	Horizontal reach-avoid with $\overline{\mathcal{B}_h}$ avoid
<code>solve_horizontal_max_distance_6d</code>	$V_{h,T}^{6D}$	6D	6D extension with obstacles

Value function interpolation uses `scipy.interpolate.RegularGridInterpolator` with linear interpolation and appropriate boundary handling (`fill_value` set to large positive values for out-of-domain queries).

3.2.4 Winning Conditions

The `winning_conditions.py` module implements:

- `compute_T_goal`: Estimates earliest time the attacker reaches its target from $\Phi_{A,\text{reach}}$.
- `compute_T_capture`: Estimates vertical capture time from Φ_z magnitude.
- `compute_T_capture_from_slices`: The paper-accurate method — iterates through saved Φ_z time slices and finds the first time at which $\Phi_z(x, t) \leq 0$.
- `check_defender_wins`: Combined check of all winning conditions.
- `get_winning_regions`: Extracts $\mathcal{W}_D$ and $\mathcal{W}_A$ masks from a value function.

3.2.5 Value Function I/O

All value functions are stored as NumPy `.npz` files with a standardized format:

```
@dataclass
class ValueFunctionData:
    values: np.ndarray      # The value function array
    grid_min: np.ndarray    # Grid lower bounds
    grid_max: np.ndarray    # Grid upper bounds
    grid_shape: tuple       # Grid dimensions
    params: dict            # Game parameters used
    description: str        # Human-readable description
```

Time slices (for T_{capture} extraction) are stored separately as `phi_z_time_slices.npz` containing the full time evolution of Φ_z .

Table 3: ROS 2 packages and their roles.

Package	Purpose
reach_avoid_bringup	Launch files, configuration YAML, dependency management
reach_avoid_sim	PX4 adapter (MAVLink $\leftrightarrow$ ROS2), ground truth relay from Gazebo
reach_avoid_controller	Defender controller implementing Algorithms 1 & 2 with value function lookup
attacker_controller	Attacker with modes: scripted waypoints, keyboard teleop, optimal escape, switchable
reach_avoid_viz	MarkerArray publisher for obstacles, target, capture zones in RViz
reach_avoid_hw	Crazyswarm2 adapter for Crazyflie drones, safety monitor

Online Control: ROS 2 Workspace

3.3.1 Package Overview

3.3.2 Defender Controller Node

The `DefenderControllerNode` runs at 50 Hz and implements the two-algorithm reach-track controller:

Algorithm 1 Vertical Reach-Track (Algorithm 1)

```

1: Input: Vertical state  $(z_D, v_{D,z}, z_A)$ , value functions  $\Phi_z, V_{z,\infty}, \mathcal{B}_z$ 
2: Compute  $z_{\text{rel}} \leftarrow z_D - z_A$ 
3: Look up  $\mathcal{B}_z(z_{\text{rel}}, v_{D,z})$ 
4: if NOT in  $\mathcal{B}_z$  then
5:    $u_z \leftarrow$  optimal reaching control from  $\nabla\Phi_z$  ▷ Mode: REACHING
6: else if deep inside  $\mathcal{B}_z$  then
7:    $u_z \leftarrow \text{PID}(z_A - z_D)$  ▷ Mode: PID
8: else
9:    $u_z \leftarrow$  optimal tracking control from  $\nabla V_{z,\infty}$  ▷ Mode: TRACKING
10: end if
11: return  $u_z$ 

```

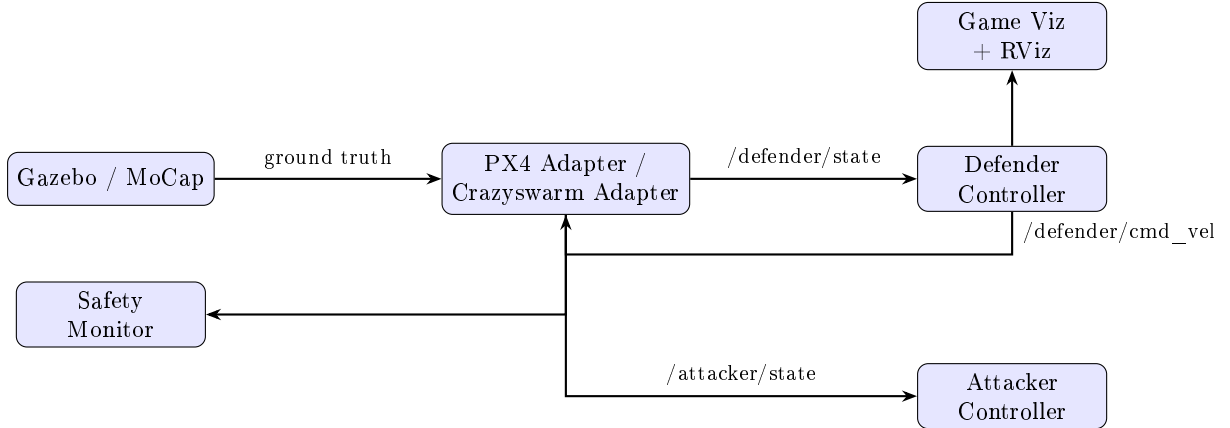
Algorithm 2 Horizontal Reach-Track-Avoid (Algorithm 2)

```

1: Input: Horizontal state  $(x_D, y_D, v_{D,x}, v_{D,y}, x_A, y_A)$ , value functions  $\Phi_h, V_{h,T}, \mathcal{B}_h$ 
2: Compute relative state  $(x_{\text{rel}}, y_{\text{rel}}, v_{D,x}, v_{D,y})$ 
3: Look up  $\mathcal{B}_h(x_{\text{rel}}, y_{\text{rel}}, v_{D,x}, v_{D,y})$ 
4: if NOT in  $\mathcal{B}_h$  then
5:    $(u_x, u_y) \leftarrow$  optimal reaching control from  $\nabla\Phi_h$  ▷ Mode: REACHING
6: else if deep inside  $\mathcal{B}_h$  then
7:    $(u_x, u_y) \leftarrow \text{PID}(x_A - x_D, y_A - y_D)$  ▷ Mode: PID
8: else
9:    $(u_x, u_y) \leftarrow$  optimal tracking control from  $\nabla V_{h,T}$  ▷ Mode: TRACKING
10: end if
11: return  $(u_x, u_y)$ 

```

3.3.3 ROS 2 Topic Architecture



Key topics:

- /defender/state (PoseStamped) — defender position from ground truth
- /attacker/state (PoseStamped) — attacker position
- /defender/cmd_vel (Twist) — velocity command to PX4/Crazyflie
- /game/status (String) — game status (CAPTURED, DEFENDER_WINNING, etc.)
- /safety/emergency (Bool) — emergency landing trigger

Configuration

All game parameters are defined in `config/game_params.yaml`:

Key parameters from game_params.yaml

```

room:
  x_min: 0.0    x_max: 45.0
  y_min: 0.0    y_max: 25.0
  z_min: 0.0    z_max: 20.0
defender:
  max_speed_horizontal: 6.0    # m/s
  max_speed_vertical: 4.0      # m/s
  k_x: 0.7    k_y: 0.7    k_z: 1.5
attacker:
  max_speed_horizontal: 3.0    # m/s
  max_speed_vertical: 2.0      # m/s
capture:
  d_h: 3.0    # horizontal capture distance (m)
  d_z: 1.0    # vertical capture distance (m)
target_region:
  x_min: 38.0    x_max: 45.0
  y_min: 10.0    y_max: 15.0
obstacles:
  - type: box
    x_min: 15.0    x_max: 20.0
    y_min: 5.0     y_max: 20.0
  
```

Three grid presets are available:

Table 4: Grid resolution presets.

Preset	Vertical 2D	Vertical 3D	Horizontal 4D/6D
dev	51×31	$24 \times 10 \times 24$	$21^2 \times 11^2$
medium	120×50	$120 \times 50 \times 120$	$43^2 \times 6^2$
paper	240×100	$240 \times 100 \times 240$	$85^2 \times 8^2$

Running the Offline Computation

Prerequisites

```
# Install the reach_avoid_game package (editable mode)
cd /workspace/reach_avoid_game
pip install -e ".[dev]"

# Verify installation
python3 -c "import reach_avoid_game; print('OK')"
python3 -c "import hj_reachability; print('hj_reachability OK')"
```

Computing Value Functions

The computation must follow the correct dependency order.

4.2.1 Step 1: Vertical Pipeline

```
# Compute V_z_inf -> B_z -> Phi_z (with B_z feedback)
python3 /workspace/reach_avoid_game/scripts/compute_vertical.py \
    --preset dev

# Output files:
# /workspace/data/value_functions/V_z_inf.npz
# /workspace/data/value_functions/B_z.npz
# /workspace/data/value_functions/phi_z.npz
# /workspace/data/value_functions/phi_z_time_slices.npz
```

4.2.2 Step 2: Horizontal Pipeline

```
# Compute V_h_T -> B_h -> phi_A_reach -> Phi_h (with B_h feedback)
python3 /workspace/reach_avoid_game/scripts/compute_horizontal.py \
    --preset dev

# Optional: include 6D extension
python3 /workspace/reach_avoid_game/scripts/compute_horizontal.py \
    --preset dev --include-6d

# Output files:
# /workspace/data/value_functions/V_h_T.npz
# /workspace/data/value_functions/B_h.npz
# /workspace/data/value_functions/phi_A_reach.npz
# /workspace/data/value_functions/phi_h.npz
# /workspace/data/value_functions/V_h_T_6d.npz (if --include-6d)
```

Note

The **dev** preset takes minutes. The **medium** preset takes tens of minutes. The **paper** preset (~ 3 hours for Φ_h alone) requires significant RAM and ideally a GPU.

Running Numerical Simulations

```
# Vertical-only simulation (Algorithm 1)
python3 /workspace/reach_avoid_game/scripts/numerical_sim.py \
    --mode vertical --T 10.0

# Combined 3D simulation (Algorithm 1 + Algorithm 2)
python3 /workspace/reach_avoid_game/scripts/numerical_sim.py \
    --mode combined --T 10.0

# Quick sanity check (1 step, verifies value function loading)
python3 /workspace/reach_avoid_game/scripts/numerical_sim.py \
    --mode combined --check-only
```

Generating Paper Figures

```
python3 /workspace/reach_avoid_game/scripts/generate_paper_figures.py \
    --output-dir /workspace/data/plots/paper_figures/ \
    --preset dev
```

This generates 10 figures corresponding to Figures 4–12 of the paper: $\mathcal{B}_z$ invariant set, Φ_z slices, vertical 3D view, $\mathcal{B}_h$ invariant set, Φ_h slices, winning regions, simulation trajectories, and a combined 3D game view.

Running Tests

```
cd /workspace/reach_avoid_game
python3 -m pytest tests/ -v

# 60 tests covering:
#   - Grid dimensions and value function shapes
#   -  $\Phi_z$  properties (monotonicity, sign conventions)
#   -  $\mathcal{B}_z$  non-emptiness and center containment
#   - Control extraction within speed limits
#   - Gradient finite-ness
#   -  $T_{\text{capture}}$  from time slices
#   - Winning region partitioning
#   - Value function I/O round-trip
```

Running the Gazebo Simulation**Prerequisites**

The Gazebo simulation requires:

- ROS2 Humble
- Gazebo Harmonic

- PX4 Autopilot SITL (Software-In-The-Loop)
- micro-XRCE-DDS-Agent (PX4 ↔ ROS 2 bridge)
- NVIDIA GPU with `nvidia-container-toolkit` (for Docker)

These are **not** installed in the development environment. Use Docker.

Docker Setup

```
# Step 1: Build the simulation Docker image
docker build -f Dockerfile.sim -t reach-avoid-sim .

# Step 2: Allow X11 display forwarding (for Gazebo GUI)
xhost +local:docker

# Step 3: Start the simulation container
docker-compose -f docker-compose.sim.yml up
```

The Docker image (`Dockerfile.sim`) installs:

- Ubuntu 22.04 base
- ROS 2 Humble Desktop (`ros-humble-desktop`)
- Gazebo Harmonic bridge (`ros-humble-ros-gz`)
- PX4 Autopilot compiled for SITL with `gz_x500` model
- micro-XRCE-DDS-Agent from source
- `px4_msgs` and `px4_ros_com` ROS 2 packages
- The full `reach_avoid_ws` workspace, pre-built with `colcon build`

The `docker-compose.sim.yml` mounts:

- `./data/value_functions` → container (read-only) for precomputed value functions
- `./config` → container (read-only) for game parameters
- X11 socket for Gazebo GUI display

Launch Commands

Once inside the Docker container:

```
# Source the workspace
source /home/simuser/ws/install/setup.bash

# Launch simulation only (Gazebo + PX4 + adapters + attacker)
ros2 launch reach_avoid_bringup simulation.launch.py \
  attacker_mode:=scripted

# Launch full game (simulation + defender controller + viz + RViz)
ros2 launch reach_avoid_bringup full_game.launch.py \
  attacker_mode:=scripted \
  value_function_dir:=/home/simuser/ws/data/value_functions/

# Launch attacker only (no Gazebo, for testing controller logic)
ros2 launch reach_avoid_bringup attacker_only.launch.py
```

Attacker modes:

- `scripted` — follows predefined waypoints toward target
- `keyboard` — manual control via `teleop_twist_keyboard`
- `optimal` — uses value function gradient for optimal escape
- `switchable` — switch between modes at runtime

Gazebo World

The simulation world (`reach_avoid_arena.sdf`) defines:

- $8 \times 8 \times 4$ m room with four walls
- Red obstacle box at $x \in [3, 4]$, $y \in [2, 6]$ m
- Green transparent target region at $x \in [6, 8]$, $y \in [3, 5]$ m
- DART physics at 1 kHz, real-time factor 1.0
- Two PX4 x500 quadrotor models (defender and attacker)

Simulation Parameters

The file `reach_avoid_bringup/config/simulation_params.yaml` configures:

- Topic names for defender/attacker state, velocity, and commands
- PX4 vehicle IDs (defender=1, attacker=2) and MAVLink ports
- Gazebo model names (`x500_defender`, `x500_attacker`)
- Control rates: PX4 heartbeat 20 Hz, ground truth 50 Hz, control 20 Hz
- Default attacker waypoints navigating around the obstacle to the target

Running on Real Drones

Safety Critical

Hardware deployment involves real flying robots. Always have a safety pilot ready. Always test with conservative speed limits first. The safety monitor will trigger emergency landing on any violation, but human oversight is essential.

Hardware Requirements

Table 5: Required hardware components.

Component	Details
Crazyflie 2.1 (x2)	One defender, one attacker. Must have unique radio addresses.
Crazyradio PA (x1)	USB radio dongle for communicating with Crazyflies.
Motion capture system	OptiTrack, Vicon, or similar. Must publish pose at ≥ 100 Hz.
Ground station PC	Ubuntu 22.04, ROS 2 Humble, Crazyswarm2 installed.
Flying arena	Minimum $4 \times 4 \times 3$ m clear space with safety nets.

Software Setup

```
# 1. Install ROS 2 Humble (if not already installed)
sudo apt install ros-humble-desktop

# 2. Install Crazyswarm2
mkdir -p ~/crazyswarm2_ws/src
cd ~/crazyswarm2_ws/src
```

```

git clone https://github.com/IMRCLab/crazyswarm2
cd ..
colcon build --symlink-install
source install/setup.bash

# 3. Build the reach_avoid_ws workspace
cd /workspace/reach_avoid_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash

# 4. Install the offline computation package
cd /workspace/reach_avoid_game
pip install -e .

# 5. Compute value functions (use medium or paper preset for real flights)
python3 scripts/compute_vertical.py --preset medium
python3 scripts/compute_horizontal.py --preset medium

```

Crazyflie Configuration

Edit `reach_avoid_bringup/config/hardware_params.yaml`:

```

crazyswarm_adapter:
  ros__parameters:
    defender_uri: "radio://0/80/2M/E7E7E7E701"
    attacker_uri: "radio://0/80/2M/E7E7E7E702"
    mocap_topic: "/mocap/rigid_bodies"
    takeoff_height: 1.0
    cmd_rate: 20.0

```

Change the URIs to match your Crazyflie radio addresses. The motion capture topic must match your MoCap system's ROS2 output.

Pre-Flight Checklist

1. Charge both Crazyflie batteries fully.
2. Verify motion capture tracking for both drones (all rigid body markers visible).
3. Place drones at known starting positions within the arena.
4. Verify radio link: `cfclient` should show both drones.
5. Confirm value function files exist in `/workspace/data/value_functions/`.
6. Clear the arena of people and obstacles not part of the experiment.
7. Brief the safety pilot on emergency procedures.

Launch Sequence

```

# Terminal 1: Source workspace
source /workspace/reach_avoid_ws/install/setup.bash

# Terminal 2: Launch the hardware game
ros2 launch reach_avoid_bringup hardware.launch.py \
  attacker_mode:=scripted \

```

```

value_function_dir:=/workspace/data/value_functions/ \
defender_uri:=radio://0/80/2M/E7E7E7E701 \
attacker_uri:=radio://0/80/2M/E7E7E7E702 \
use_rviz:=true

# Terminal 3: Arm the safety monitor
ros2 service call /safety/arm std_srvs/srv/Trigger

# Terminal 4: Command takeoff
ros2 service call /hw/takeoff std_srvs/srv/Trigger

# The defender controller will automatically begin tracking the attacker
# once both drone states are being received.

# Emergency landing (any terminal):
ros2 service call /hw/land std_srvs/srv/Trigger

```

Safety Monitor

The safety monitor node (`reach_avoid_hw/safety_monitor_node.py`) runs at 50 Hz and checks:

Table 6: Safety checks and their parameters.

Check	Condition	Default
Geofence	Position within room bounds $\pm$ margin	0.5 m margin
Altitude floor	$z > z_{\min}$	0.3 m
Altitude ceiling	$z < z_{\max} - \text{margin}$	0.5 m margin
Defender speed	$\ v\ < U_D \times 1.1$	6.6 m/s
Attacker speed	$\ v\ < U_A \times 1.1$	3.3 m/s
Inter-drone distance	$\ p_D - p_A\ > d_{\min}$	0.5 m
State watchdog	State received within timeout	1.0 s

Fail-safe behavior: The safety monitor defaults to emergency landing on:

- Any safety violation
- State data timeout (communication loss)
- Any internal error (exception during safety check)

The emergency state **latches** — once triggered, the monitor stays in emergency until manually rearmed via the `/safety/arm` service.

Crazyswarm2 Adapter

The adapter node bridges the reach-track controller to the Crazyflie hardware:

- Subscribes to `/defender/cmd_vel` (Twist) from the defender controller.
- Forwards velocity commands to the Crazyflie via `cmdVelocityWorld`.
- Reads motion capture poses from `/mocap/defender` and `/mocap/attacker`.
- Publishes drone states (`/defender/state`, `/attacker/state`) and velocity estimates.
- Provides takeoff (`/hw/takeoff`) and landing (`/hw/land`) services.

Velocity estimation is done via finite differences on motion capture position data. For more accurate velocity, consider using the Crazyflie’s onboard state estimator.

Tuning for Real Hardware

Warning

The value functions computed with the paper’s speed parameters ($U_D^h = 6 \text{ m/s}$) may be too aggressive for indoor Crazyflie flights. Recommended adjustments for initial testing:

- Reduce `max_speed_horizontal` to 1.0–2.0 m/s
- Reduce `max_speed_vertical` to 0.5–1.0 m/s
- Increase `min_inter_drone_distance` to 1.0 m
- Recompute value functions with the adjusted parameters

The proportional gains (k_x, k_y, k_z) model the Crazyflie’s velocity response. Measure the actual step response of your drone and adjust these gains to match. A gain that is too high will cause the value function to overestimate the drone’s agility; too low will make it too conservative.

Troubleshooting

Phi_z is all positive (no winning region)

On coarse grids (dev preset), the invariant capture set $\mathcal{B}_z$ may be very small, causing Φ_z to remain positive everywhere. Use the `medium` or `paper` preset for meaningful results.

T_capture returns infinity

Same root cause as above. The time slices never reach ≤ 0 on coarse grids because $\mathcal{B}_z$ is too small as a target set.

Gazebo crashes immediately

Check that the NVIDIA container toolkit is installed and X11 forwarding is configured. Run `xhost +local:docker` before `docker-compose up`.

PX4 SITL does not connect

Ensure MicroXRCEAgent is running on UDP port 8888. The PX4 instance must be configured with the correct `PX4_SIM_MODEL=gz_x500`.

Defender hovers without moving

The controller falls back to hover when value functions are not loaded. Check that the `value_function_dir` parameter points to a directory containing all required `.npz` files.

Safety monitor triggers immediately

Verify that motion capture is publishing valid poses. The watchdog triggers emergency if no state is received within 1 s.

RegularGridInterpolator bounds error

States are clamped to grid bounds before interpolation. If you see this error, a clamping step is missing — check that `np.clip` is applied before calling the interpolator.

Value function NaN or Inf after solve

Verify the initial value function (SDF) is finite everywhere. Check that grid domain bounds match the room dimensions in `game_params.yaml`.

References

1. M. Bui, S. Monckton, and M. Chen, “Reach-Avoid Differential Game with Reachability Analysis for UAVs — A Decomposition Approach,” *arXiv preprint arXiv:2512.22793*, Dec. 2025.

2. `hj_reachability`: https://github.com/StanfordASL/hj_reachability — JAX-based Hamilton-Jacobi reachability library (v0.7.0).
3. PX4 Autopilot: <https://github.com/PX4/PX4-Autopilot> — Open-source flight controller.
4. CrazySwarm2: <https://github.com/IMRCLab/crazyswarm2> — ROS2 driver for Crazyflie drones.
5. Gazebo: <https://gazebo.org/> — Robotics simulation platform (Harmonic release).